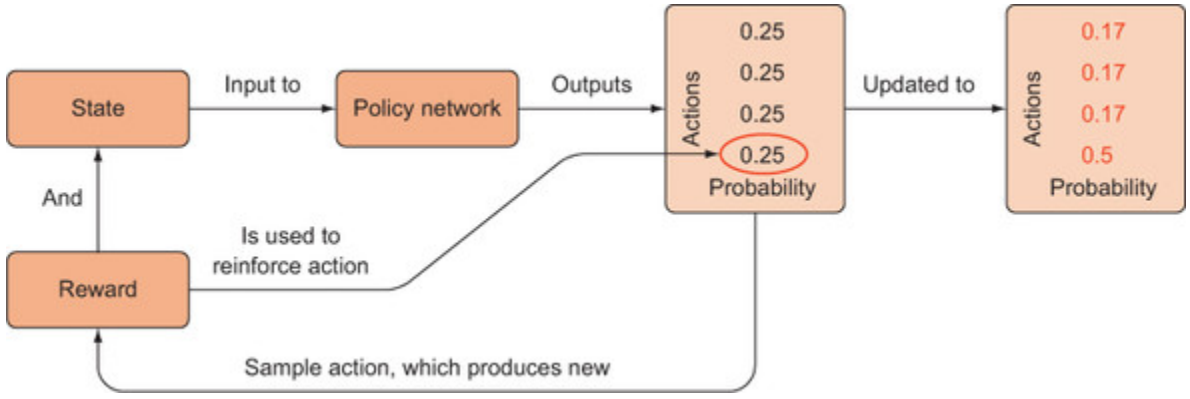
more confident in predicting action 3 as the highest probability action to take.

A naive approach might be to make a target action distribution, [0, 0, 0, 1], so that our gradient descent will move the probabilities from [0.25, 0.25, 0.25, 0.25] close to [0, 0, 0, 1], maybe ending up as [0.167, 0.167, 0.167, 0.5] (see figure 4.6). This is something we often do in the supervised learning realm, when we are training a softmax-based image classifier. But in that case, there is a single correct classification for an image, and there is no temporal association between each prediction. In our RL case, we want more control over how we make these updates. First, we want to make small, smooth updates because we want to maintain some stochasticity in our action sampling to adequately explore the environment. Second, we want to be able to weight how much we assign credit to each action for earlier actions. Let's review some more notation before diving into these two problems.

Figure 4.6. Once an action is sampled from the policy network's probability distribution, it produces a new state and reward. The reward signal is used to reinforce the action that was taken, that is, it increases the probability of that action given the state if the reward is positive, or it decreases the probability if the reward is negative. Notice that we only received information about action 3 (element 4), but since the probabilities must sum to 1, we have to lower the probabilities of the other actions.



Recall that our policy network is typically denoted π_θ when we are running it forward (i.e., using it to produce action probabilities), because we think of the network parameters, θ, as being fixed and the input state is what varies. Hence, calling $\pi_\theta(s)$ for some state s will return a probability distribution over the possible actions, given a fixed set of parameters. When we are